NGHIÊN CỨU MÔ HÌNH HÓA SAT ENCODING CHO RÀNG BUỘC CARDINALITY (AMO/ALO) VÀ ĐÁNH GIÁ THỰC NGHIỆM SO SÁNH GIẢI SUDOKU $N \times N$ VỚI CÁC PHƯƠNG PHÁP CHÍNH XÁC (CP, ILP/MIP)

Nguyễn Minh Sơn[a,*] (Nghiên cứu sinh thực hiện)

[a] *Khoa Khoa học Máy tính, Trường Công nghệ Thông tin và Truyền thông, Học kỳ I (2026–2027)*

ARTICLE INFO

Keywords:
Sudoku Solver
SAT Encoding
AMO Constraints
Constraint Programming
Integer Linear Programming
PySAT
Glucose3
OR-Tools
Gurobi
CPLEX

Tóm tắt nội dung

Bài toán Sudoku tổng quát trên ma trận kích thước $N \times N$ ($4 \times 4, 9 \times 9, 16 \times 16$) là bài toán thuộc lớp NP-complete kinh điển, mang tính chất đại diện tiêu biểu cho các bài toán thỏa mãn ràng buộc (CSP) và giải đố logic. Trọng tâm của bài báo này là nghiên cứu, thiết kế và đánh giá thực nghiệm các kỹ thuật mã hóa mệnh đề logic (SAT Encoding) cho các ràng buộc số lượng (Cardinality Constraints), đặc biệt là ràng buộc At-Most-One (AMO), At-Least-One (ALO) và Exactly-One (EO). Nghiên cứu triển khai thực nghiệm 5 kỹ thuật mã hóa AMO chuyên sâu: *Pairwise (Naive)*, *Binary (Bitwise)*, *Commander (Klieber & Kwon, 2007)*, *2-Product (Jingchao Chen, 2010)*, và *Sequential Counter (Carsten Sinz, 2005)*. Hiệu năng của SAT Solver (*Glucose3* thông qua thư viện PySAT) được đặt trong mối tương quan so sánh đối chuẩn toàn diện với các công cụ giải chính xác hàng đầu thế giới thuộc hai trường phái Constraint Programming (CP) và Integer Linear Programming (ILP/MIP): *Google OR-Tools CP-SAT*, *IBM CPLEX CP Optimizer*, *Gurobi Optimizer (MIP)*, *IBM CPLEX MIP*, và *PuLP (CBC)*. Thực nghiệm được tiến hành trên tập dữ liệu chuẩn hóa gồm 299 câu đố với đa dạng kích thước ($4 \times 4, 9 \times 9, 16 \times 16$) và 4 cấp độ khó (*Easy, Medium, Hard, Expert*) thu thập từ các kho dữ liệu quốc tế như Peter Norvig Benchmark, Sudoku Exchange Puzzle Bank, KaggleHub và Hugging Face. Kết quả thực nghiệm làm sáng tỏ quy luật sinh biến phụ, số lượng mệnh đề CNF, tính hiệu quả của cơ chế lan truyền đơn vị (unit propagation) và khẳng định lợi thế về tốc độ của phương pháp SAT cho bài toán Sudoku so với các bộ giải tối ưu hóa truyền thống (CP, ILP/MIP).

1. Giới thiệu (Introduction)

Bài toán Sudoku là một trò chơi giải đố sắp xếp chữ số tổ hợp đã thu hút sự chú ý sâu sắc của cộng đồng Khoa học máy tính và Trí tuệ nhân tạo (AI). Trên bảng kích thước chuẩn 9×9 chia thành 9 khối con 3×3, người chơi cần điền các chữ số từ 1 đến 9 vào các ô trống sao cho mỗi chữ số xuất hiện đúng một lần duy nhất trên mỗi hàng, mỗi cột và mỗi khối con. Mở rộng bài toán sang kích thước tổng quát $N \times N$ (với $N = k^2$, khối con kích thước $k \times k$), bài toán Sudoku đã được chứng minh là thuộc lớp bài toán **NP-complete** Norvig (2006).

Trong lý thuyết tính toán, SAT (Boolean Satisfiability Problem) là bài toán NP-complete đầu tiên được chứng minh theo định lý Cook-Levin. Nhờ vào sự phát triển vượt bậc của các thuật toán CDCL (Conflict-Driven Clause Learning) trong hai thập kỷ qua, các bộ giải SAT hiện đại (như Glucose Audemard and Simon (2009), Cadical, Kissat) có khả năng giải quyết các bài toán công nghiệp chứa hàng triệu biến và mệnh đề chỉ trong vài giây. Một trong những thách thức quan trọng nhất khi áp dụng SAT vào thực tế là **kỹ thuật mã hóa (SAT Encoding)** các ràng buộc bài toán sang dạng chuẩn hội CNF (Conjunctive Normal Form), đặc biệt là các ràng buộc số lượng (*Cardinality Constraints*): At-Least-One (ALO - có ít nhất một biến đúng), At-Most-One (AMO - có nhiều nhất một biến đúng) và Exactly-One (EO - có đúng một biến đúng).

Đồng thời, bên cạnh tiếp cận bằng SAT, bài toán Sudoku còn có thể được mô hình hóa và giải quyết bằng các phương pháp toán học chính xác khác như:

*Tác giả liên hệ chính.
student@university.edu.vn (N.M. Sơn)
ORCID(s):

- **Constraint Programming (CP)**: Khai thác các ràng buộc toàn cục như `AllDifferent` thông qua lan truyền miền giá trị cục bộ (domain filtering), tiêu biểu là Google OR-Tools CP-SAT Perron and Furnon (2024) và IBM ILOG CPLEX CP Optimizer IBM Corporation (2024).
- **Integer Linear Programming (ILP / MIP)**: Sử dụng quy hoạch tuyến tính nguyên với các biến nhị phân $\{0, 1\}$, giải quyết bằng phương pháp nhánh cận (Branch-and-Bound) kết hợp vết cắt phẳng (Cutting Planes), tiêu biểu là Gurobi Optimizer Gurobi Optimization, LLC (2024), IBM CPLEX MIP, và CBC Solver.

Mục tiêu của bài báo này là:

1. Xây dựng hệ thống giải Sudoku tổng quát $N \times N$ hỗ trợ đầy đủ 5 kỹ thuật mã hóa AMO tiên tiến nhất trong lý thuyết SAT.
2. Triển khai thực nghiệm so sánh đối chuẩn SAT Solver với 5 Solver chính xác hàng đầu thế giới (CP-SAT, CPLEX CP, Gurobi, CPLEX MIP, ILP/CBC).
3. Đánh giá hiệu năng toàn diện trên bộ dữ liệu 299 câu đố chuẩn hóa với đa dạng kích thước ($4 \times 4, 9 \times 9, 16 \times 16$) và 4 cấp độ khó.

2. Mô hình hóa bài toán Sudoku bằng SAT (SAT Formulation)

Xét bảng Sudoku kích thước $N \times N$ với các khối con kích thước $k_r \times k_c$ ($N = k_r \times k_c$). Mỗi ô trong bảng được định vị bởi tọa độ (r, c) với $0 \le r, c < N$. Các giá trị điền vào nhận giá trị $d \in \{1, 2, \dots, N\}$.

2.1. Biểu diễn biến mệnh đề

Ta định nghĩa biến Boole bậc ba $x_{r,c,d}$:

$$x_{r,c,d} = \begin{cases} 1 & \text{nếu ô } (r, c) \text{ nhận chữ số } d \\ 0 & \text{ngược lại} \end{cases} \tag{1}$$

Số lượng biến Boole ban đầu cho bảng kích thước $N \times N$ là $N \times N \times N = N^3$ biến. Trong cài đặt mã nguồn, chỉ số biến (1-based index) được ánh xạ theo công thức song ánh:

$$\text{ID}(r, c, d) = N^2 \cdot r + N \cdot c + d \quad (1 \le \text{ID} \le N^3) \tag{2}$$

2.2. Hệ thống ràng buộc của Sudoku

Một cấu hình Sudoku hợp lệ phải thỏa mãn đồng thời 5 nhóm ràng buộc logic sau:

1. **Ràng buộc ô (Cell Constraints)**: Mỗi ô (r, c) phải nhận *đúng một* chữ số từ 1 đến N:

$$\text{Exactly-One}\Big(\{x_{r,c,d} \mid 1 \le d \le N\}\Big), \quad \forall 0 \le r, c < N \tag{3}$$

2. **Ràng buộc hàng (Row Constraints)**: Mỗi chữ số d phải xuất hiện *đúng một* lần trên mỗi hàng r:

$$\text{Exactly-One}\Big(\{x_{r,c,d} \mid 0 \le c < N\}\Big), \quad \forall 0 \le r < N,\ 1 \le d \le N \tag{4}$$

3. **Ràng buộc cột (Column Constraints)**: Mỗi chữ số d phải xuất hiện *đúng một* lần trên mỗi cột c:

$$\text{Exactly-One}\Big(\{x_{r,c,d} \mid 0 \le r < N\}\Big), \quad \forall 0 \le c < N,\ 1 \le d \le N \tag{5}$$

4. **Ràng buộc khối con (Block/Box Constraints)**: Mỗi chữ số d phải xuất hiện *đúng một* lần trong mỗi khối (br, bc):

$$\text{Exactly-One}\Big(\{x_{r,c,d} \mid r \in \text{block}_r(br), c \in \text{block}_c(bc)\}\Big) \tag{6}$$

5. **Ràng buộc gợi ý ban đầu (Clue / Given Constraints)**: Nếu ô (r, c) đã có giá trị cho trước $v \in \{1, \dots, N\}$, mệnh đề đơn nguyên (unit clause) được bổ sung:

$$(x_{r,c,v}) \quad \forall (r, c, v) \in \text{Clues} \tag{7}$$

Một ràng buộc Exactly-One (EO) trên tập biến $X = \{x_1, \dots, x_n\}$ được phân rã thành:

$$\text{EO}(X) \iff \text{At-Least-One}(X) \land \text{At-Most-One}(X) \tag{8}$$

Ràng buộc **At-Least-One (ALO)** được biểu diễn đơn giản bằng 1 mệnh đề hội:

$$\text{ALO}(X) = (x_1 \lor x_2 \lor \dots \lor x_n) \tag{9}$$

Trong khi đó, việc mã hóa ràng buộc **At-Most-One (AMO)** có ảnh hưởng quyết định đến quy mô của file CNF và thời gian giải của SAT Solver.

3. Các kỹ thuật mã hóa At-Most-One (AMO Encodings)

Cho tập gồm n biến Boole $X = \{x_1, x_2, \dots, x_n\}$. Ràng buộc At-Most-One phát biểu rằng có nhiều nhất một biến trong X nhận giá trị True: $\sum_{i=1}^{n} x_i \leq 1$. Trong nghiên cứu này, 5 kỹ thuật mã hóa AMO chuyên sâu được triển khai và đánh giá chi tiết:

3.1. Pairwise Encoding (Naive Approach)

Phương pháp tổ hợp cặp đôi cổ điển phát biểu rằng hai biến bất kỳ trong X không thể đồng thời nhận giá trị True:

$$\text{AMO}_{\text{pair}}(X) = \bigwedge_{1 \leq i < j \leq n} (\neg x_i \lor \neg x_j) \tag{10}$$

- **Số biến phụ**: 0.
- **Số mệnh đề**: $\binom{n}{2} = \frac{n(n-1)}{2} = O(n^2)$.
- **Đặc điểm**: Dễ cài đặt, không làm tăng không gian biến tìm kiếm nhưng số mệnh đề bùng nổ theo hàm bậc hai khi n tăng lớn.

3.2. Binary Encoding (Bitwise Approach)

Được chuẩn hóa bởi Frisch et al. (2008), phương pháp này đánh số nhị phân cho mỗi biến x_i với $m = \lceil \log_2 n \rceil$ biến phụ $b_1, b_2, \dots, b_m$. Mỗi biến x_i được gán với biểu diễn nhị phân của chỉ số i. Nếu $x_i = 1$, các biến phụ phải nhận chính xác trạng thái nhị phân của i:

$$\text{AMO}_{\text{bin}}(X) = \bigwedge_{i=1}^{n} \bigwedge_{j=1}^{m} \left(\neg x_i \lor \ell(i,j) \right) \tag{11}$$

trong đó $\ell(i,j) = b_j$ nếu bit thứ j của $(i-1)$ là 1, và $\ell(i,j) = \neg b_j$ nếu bit thứ j là 0.

- **Số biến phụ**: $\lceil \log_2 n \rceil = O(\log n)$.
- **Số mệnh đề**: $n\lceil \log_2 n \rceil = O(n \log n)$.
- **Đặc điểm**: Tiết kiệm biến phụ và mệnh đề đáng kể khi n rất lớn.

3.3. Commander Variable Encoding

Kỹ thuật do Klieber and Kwon (2007) đề xuất. Ý tưởng chính là phân hoạch n biến thành các nhóm con kích thước k (thường chọn $k = 3$). Mỗi nhóm được chỉ định một biến chỉ huy (*commander variable*) c_g:

1. Bên trong mỗi nhóm con, áp dụng Pairwise AMO: $\bigwedge_{u<v}(\neg u \lor \neg v)$.
2. Ràng buộc biến phần tử với biến chỉ huy: $x \to c_g$ $(\neg x \lor c_g)$.
3. Ràng buộc biến chỉ huy với nhóm: $c_g \to \bigvee_{x \in g} x$ $(\neg c_g \lor \bigvee x)$.
4. Áp dụng đệ quy ràng buộc AMO trên tập biến chỉ huy $\{c_1, c_2, \dots\}$.

- **Số biến phụ**: $O(n/k) = O(n)$.
- **Số mệnh đề**: $O(n)$ (tuyến tính cả về biến và mệnh đề).

Bảng 1
So sánh đặc tính lý thuyết của 5 phương pháp mã hóa At-Most-One (AMO).

Phương pháp AMO	Tác giả / Nguồn	Số biến phụ	Số mệnh đề	Đặc tính Unit Propagation
Pairwise (Naive)	Cổ điển	0	$\frac{n(n-1)}{2} = O(n^2)$	Đạt Arc-consistency
Binary (Bitwise)	Frisch et al. (2008)	$\lceil \log_2 n \rceil$	$n\lceil \log_2 n \rceil = O(n \log n)$	Không bảo toàn Arc-consistency
Commander	Klieber and Kwon (2007)	$O(n/k)$	$O(n)$	Đạt Arc-consistency
2-Product	Chen (2010)	$2\lceil \sqrt{n} \rceil = O(\sqrt{n})$	$2n + \binom{p}{2} + \binom{q}{2} = O(n)$	Đạt Arc-consistency
Sequential Counter	Sinz (2005)	$n - 1$	$3n - 4 = O(n)$	Bảo toàn Arc-consistency tối ưu

3.4. 2-Product Encoding (Jingchao Chen, 2010)

Phương pháp 2-Product được Chen (2010) đề xuất, bố trí n biến vào một lưới 2 chiều kích thước $p \times q$ sao cho $p \times q \geq n$, với $p = \lceil \sqrt{n} \rceil$ và $q = \lceil n/p \rceil$. Ta giới thiệu p biến hàng $r_1, \ldots, r_p$ và q biến cột $c_1, \ldots, c_q$. Mỗi biến $x_{i,j}$ tương ứng với ô ở hàng i, cột j:

$$x_{i,j} \to r_i \iff (\neg x_{i,j} \vee r_i) \tag{12}$$

$$x_{i,j} \to c_j \iff (\neg x_{i,j} \vee c_j) \tag{13}$$

$$\text{AMO}(r_1, \ldots, r_p) \wedge \text{AMO}(c_1, \ldots, c_q) \tag{14}$$

- **Số biến phụ**: $p + q = 2\lceil \sqrt{n} \rceil = O(\sqrt{n})$.
- **Số mệnh đề**: $2n + \binom{p}{2} + \binom{q}{2} = O(n)$.
- **Đặc điểm**: Mã hóa rất nhanh (tuyến tính theo số biến n), cấu trúc lưới 2 chiều thanh lịch và số biến phụ chỉ tăng theo căn bậc hai của n.

3.5. Sequential Counter Encoding

Được Sinz (2005) đề xuất, sử dụng một chuỗi biến đếm $s_1, s_2, \ldots, s_{n-1}$ đại diện cho trạng thái tích lũy của chuỗi tiền tố:

$$(\neg x_1 \vee s_1) \tag{15}$$

$$(\neg x_i \vee s_i), \quad (\neg s_{i-1} \vee s_i), \quad (\neg x_i \vee \neg s_{i-1}) \quad \forall 2 \leq i \leq n-1 \tag{16}$$

$$(\neg x_n \vee \neg s_{n-1}) \tag{17}$$

- **Số biến phụ**: $n - 1$.
- **Số mệnh đề**: $3n - 4 = O(n)$.
- **Đặc tính lý thuyết nổi bật**: Kỹ thuật này duy trì tính **Arc-consistency** thông qua lan truyền đơn vị (*Unit Propagation*). Nếu một biến $x_i = 1$, solver sẽ lập tức suy diễn toàn bộ các biến còn lại phải bằng 0 mà không cần thực hiện thêm bước phân nhánh tìm kiếm (branching).

4. Các phương pháp giải đối chuẩn khác (Alternative Approaches)

Để có đánh giá khách quan toàn diện theo yêu cầu đề bài, nghiên cứu thiết lập mô hình hóa Sudoku trên 5 bộ giải chính xác thuộc các nhánh CP và ILP/MIP:

4.1. Google OR-Tools CP-SAT Solver

CP-SAT là bộ giải Constraint Programming kết hợp SAT hiện đại phát triển bởi Google Perron and Furnon (2024). Trong CP-SAT, Sudoku được mô hình hóa tự nhiên bằng mảng 2 chiều các biến nguyên $y_{r,c} \in \{1, 2, \dots, N\}$. Các ràng buộc hàng, cột và khối con được biểu diễn thông qua ràng buộc toàn cục:

$$\texttt{AddAllDifferent}\Big(\{y_{r,c} \mid c = 0 \dots N-1\}\Big) \tag{18}$$

CP-SAT tự động kích hoạt thuật toán lọc miền giá trị dựa trên đồ thị ghép đôi cực đại (Hopcroft-Karp / Regin's filtering algorithm) kết hợp bộ máy SAT CDCL bên dưới.

4.2. IBM ILOG CPLEX CP Optimizer

Bộ giải CP chuyên dụng của IBM khai báo biến kiểu `integer_var(1, N)` và áp dụng hàm ràng buộc `all_diff()`. CP Optimizer sử dụng kỹ thuật suy diễn miền giá trị liên tục kết hợp tìm kiếm cục bộ (Large Neighborhood Search) và nhánh cận.

4.3. Integer Linear Programming (ILP/MIP): Gurobi, CPLEX MIP & PuLP/CBC

Trong mô hình Quy hoạch tuyến tính nguyên:

- Biến quyết định nhị phân: $z_{r,c,d} \in \{0, 1\}$.
- Hàm mục tiêu: Tìm nghiệm thỏa mãn ràng buộc (Hàm mục tiêu bằng 0 hoặc hằng số): min 0.
- Hệ thống ràng buộc đẳng thức tuyến tính:

$$\sum_{d=1}^{N} z_{r,c,d} = 1 \quad \forall r, c \tag{19}$$

$$\sum_{c=0}^{N-1} z_{r,c,d} = 1 \quad \forall r, d \tag{20}$$

$$\sum_{r=0}^{N-1} z_{r,c,d} = 1 \quad \forall c, d \tag{21}$$

$$\sum_{r \in \text{block}} \sum_{c \in \text{block}} z_{r,c,d} = 1 \quad \forall \text{block}, d \tag{22}$$

Mô hình này được nạp trực tiếp vào **Gurobi Optimizer** (thương mại hàng đầu), **IBM CPLEX MIP** và solver mã nguồn mở **CBC** (thông qua giao tiếp PuLP).

5. Kết quả thực nghiệm và Thảo luận (Experimental Evaluation)

5.1. Môi trường thực nghiệm & Bộ dữ liệu Benchmark

- **Phần cứng**: Intel Core i5-1235U (thế hệ 12), 16 GB RAM, hệ điều hành Windows x64.
- **Môi trường phần mềm**: Python 3.14, thư viện PySAT (python-sat 1.9), OR-Tools 9.15, Gurobi 13.0, CPLEX 22.2 và PuLP 3.3.
- **Dữ liệu Benchmark**: 299 câu đố chuẩn hóa được lưu trữ tại `data/benchmark_dataset.json`:
  - Bảng chuẩn 9 × 9: 279 câu đố với 4 cấp độ khó (*19 Easy, 77 Medium, 99 Hard, 84 Expert*). Đây cũng là tập dữ liệu chính dùng cho các Bảng 2, 3 và các Hình 1–3.
  - Bảng nhỏ 4 × 4: 10 câu đố (Mini-Sudoku).
  - Bảng lớn 16 × 16 (Hexadoku): 10 câu đố với tỷ lệ gợi ý ban đầu biến thiên từ 18% đến 55%.

Bảng 2
Quy mô trung bình số biến và mệnh đề sinh ra giữa 5 phương pháp AMO trên Sudoku 9×9 (279 câu đố) và 16×16 (10 câu đố).

Phương pháp AMO	Sudoku 9x9		Sudoku 16x16	
	Số biến	Số Clauses	Số biến	Số Clauses
Pairwise (Naive)	729	12,014	4,096	123,987
Binary (Bitwise)	2,025	12,014	8,192	66,643
Commander	1,701	8,126	11,264	52,307
2-Product	2,673	8,126	12,288	46,163
Sequential Counter	3,321	7,802	19,456	46,163

5.2. Đánh giá quy mô mô hình SAT Encodings (Variables vs Clauses)

Bảng 2 và Hình 1 tổng hợp quy mô mô hình CNF sinh ra trên bảng Sudoku 9×9 và ma trận lớn 16×16.

Phân tích:

- *Về số lượng mệnh đề (Clauses)*: Phương pháp Pairwise có số lượng mệnh đề lớn nhất và bùng nổ theo hàm bậc hai ($12,014$ clauses trên 9×9 và vọt lên $123,987$ clauses trên 16×16). Ở kích thước 9×9, Binary cho số mệnh đề đúng bằng Pairwise do $\binom{9}{2} = 36 = 9\lceil \log_2 9 \rceil$; tuy nhiên khi N tăng lên 16, ưu thế tiệm cận $O(n \log n)$ của Binary thể hiện rõ và số mệnh đề giảm còn $66,643$ (khoảng -46%). Các kỹ thuật tuyến tính Commander, 2-Product và Sequential Counter giảm mạnh hơn nữa, khoảng **32–35%** trên 9×9 và **58–63%** trên 16×16 so với Pairwise, kiểm soát chặt chẽ hiện tượng bùng nổ mệnh đề khi N tăng.

- *Về thời gian mã hóa và giải SAT*: Thời gian mã hóa CNF dao động từ 2.8 ms đến 6.9 ms, trong đó **2-Product** nhanh nhất; còn thời gian giải trên Glucose3 cực kỳ ngắn, chỉ từ 0.4 ms đến 0.5 ms cho mọi phương pháp nhờ cơ chế lan truyền đơn vị. So với thời gian giải, chi phí mã hóa chiếm phần lớn tổng thời gian, vì vậy các kỹ thuật có cấu trúc tuyến tính (tiêu biểu là 2-Product) mang lại tổng thời gian xử lý nhỏ nhất.

5.3. Phân tích độ nhạy cảm theo cấp độ khó (Difficulty Analysis)

Hình 2 biểu diễn tổng thời gian (mã hóa + giải) trung bình của 5 phương pháp SAT Encoding qua 4 cấp độ khó: *Easy* → *Medium* → *Hard* → *Expert*.

Nhận xét:

- **2-Product** là phương pháp ổn định và nhanh nhất ở *mọi* cấp độ khó, dao động khoảng 2.8–4.1 ms, nhờ chi phí mã hóa thấp nhất và chỉ sinh thêm $2\lceil \sqrt{n} \rceil$ biến phụ.

- **Sequential Counter** có tổng thời gian cao nhất (khoảng 6.5–8.6 ms) vì phải sinh $n - 1$ biến đếm cho mỗi ràng buộc Exactly-One, khiến pha mã hóa trở thành nút cổ chai, dù số mệnh đề sinh ra lại ít nhất.

- Thời gian đạt đỉnh tại cấp độ *Hard* rồi giảm nhẹ ở *Expert*; biên độ biến thiên giữa các cấp độ chỉ khoảng 2–3 ms và không xuất hiện hiện tượng bùng nổ thời gian (runaway runtime). Điều này cho thấy số lượng gợi ý ban đầu ảnh hưởng không đáng kể tới chi phí xử lý, phần lớn chênh lệch đến từ dao động của pha mã hóa (xem phân tích min-of-60 trên từng câu đố đại diện tại Mục 6).

- Thời gian giải thuần túy trên Glucose3 rất nhỏ, trung bình chỉ 0.4–0.5 ms trên toàn tập cho mọi phương pháp, minh chứng cho hiệu quả của cơ chế lan truyền đơn vị trong các encoding tuyến tính. Tuy nhiên, khi đo trên từng câu đố đại diện, phần thời gian này tăng dần theo cấp độ khó do số ô gợi ý giảm (chi tiết tại Mục 6).

5.4. So sánh đối chuẩn hiệu năng giữa SAT Solver và các Solvers (CP, ILP/MIP)

Bảng 3 và Hình 3 minh họa kết quả đo lường thời gian thực thi trung bình giữa các công cụ giải khác nhau trên cùng tập dữ liệu chuẩn hóa.

Thảo luận chuyên sâu:

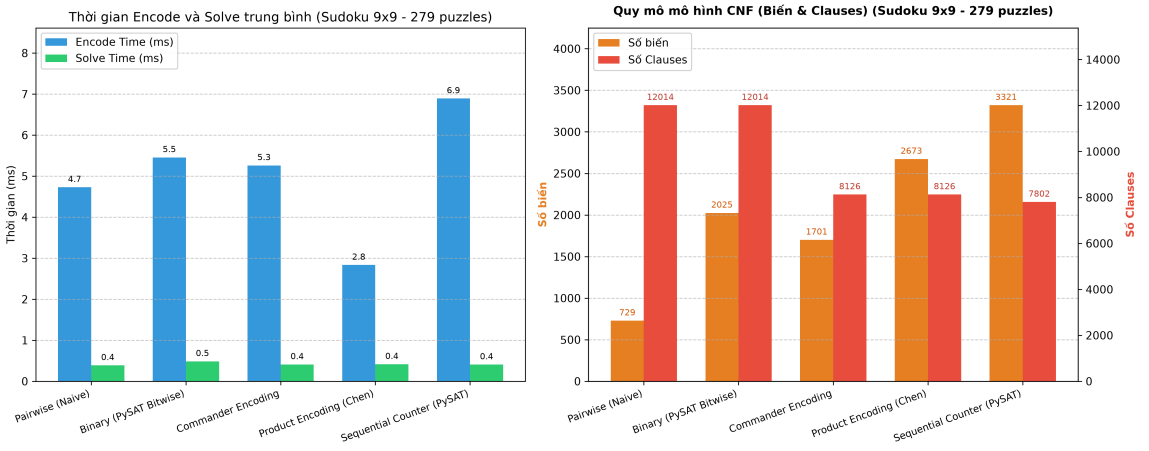
1. **SAT Solver dẫn đầu về tốc độ**: Với thời gian trung bình 10.7–16.9 ms (trung bình khoảng 12.7 ms), xét theo encoding SAT nhanh nhất (10.67 ms), SAT Solver (Glucose3) nhanh hơn Gurobi MIP (17.38 ms) khoảng **1.6**

Bảng 3
So sánh thời gian giải trung bình giữa các Solver trên tập 279 câu đố Sudoku 9×9 chuẩn hóa.

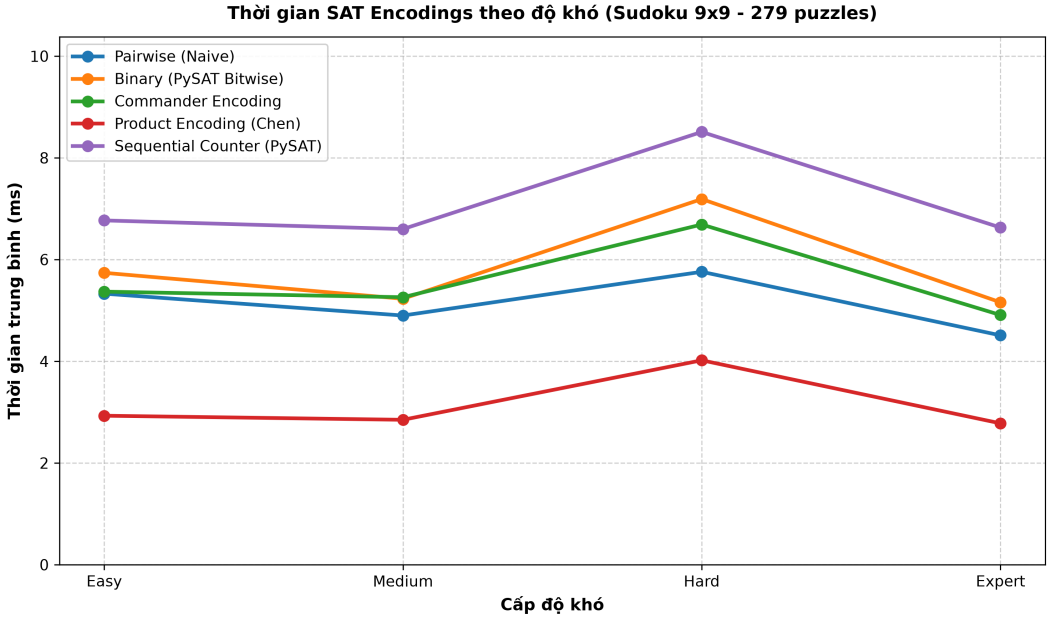
Bộ giải (Solver Engine)	Trường phái	Thời gian TB (ms)	Tỷ lệ thành công
SAT (Glucose3 - Commander)	SAT	**10.67**	100%
SAT (Glucose3 - Product)	SAT	**10.90**	100%
SAT (Glucose3 - SeqCounter)	SAT	**11.94**	100%
SAT (Glucose3 - Pairwise)	SAT	**13.21**	100%
SAT (Glucose3 - Binary)	SAT	**16.91**	100%
Gurobi Optimizer	MIP	17.38	100%
Google OR-Tools CP-SAT	CP / SAT Hybrid	19.01	100%
IBM CPLEX MIP	MIP	25.50	100%
IBM CPLEX CP Optimizer	CP	44.73	100%
ILP (PuLP / CBC)	ILP / Branch-and-Cut	147.10	100%

lần, nhanh hơn CPLEX MIP (25.50 ms) khoảng **2.4 lần**, nhanh hơn CPLEX CP Optimizer (44.73 ms) khoảng **4.2 lần** và nhanh hơn solver mã nguồn mở CBC (147.10 ms) tới khoảng **13.8 lần**. Lý do cốt lõi là bản chất rời rạc thuần túy của Sudoku rất tương thích với cấu trúc biểu diễn nhị phân của SAT cùng cơ chế 2-watched literals của các solver CDCL.

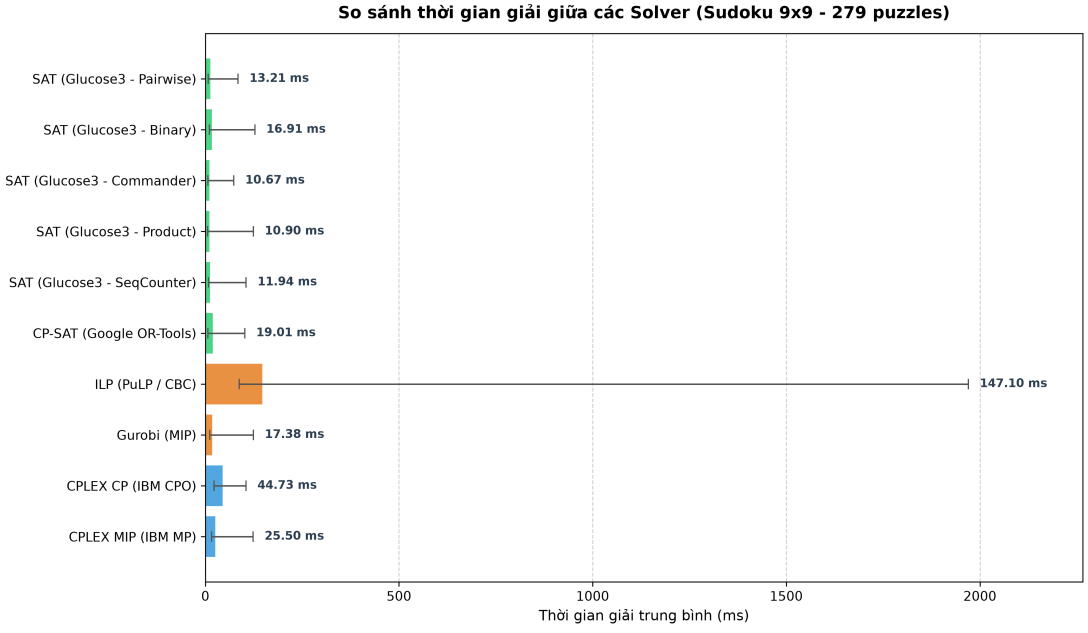
2. **Hiệu năng của CP-SAT (Google OR-Tools)**: CP-SAT khá gần với SAT thuần (19.01 ms), chỉ chậm hơn khoảng 1.5 lần so với SAT nhanh nhất. Điều này hợp lý về mặt lý thuyết vì kiến trúc bên dưới của CP-SAT thực chất là một bộ giải SAT CDCL được mở rộng để hỗ trợ các biến nguyên và bộ lọc lan truyền ràng buộc.
3. **Vị thế của Gurobi (MIP)**: Gurobi cho kết quả rất cạnh tranh (17.38 ms), gần bằng các encoding SAT chậm hơn, cho thấy các bộ giải MIP thương mại đã tối ưu rất tốt cho lớp bài toán ràng buộc chặt này.
4. **Hạn chế của các bộ giải thuần ILP/MIP**: Các bộ giải MIP xử lý bài toán thông qua việc giải bài toán nới lỏng quy hoạch tuyến tính liên tục (LP Relaxation) bằng phương pháp Simplex hoặc Điểm trong (Interior Point) trước khi cắt nhánh nguyên. Trong bài toán Sudoku, ma trận ràng buộc không đạt tính đơn mô-đun toàn phần (unimodular), nên nghiệm LP nới lỏng thường mang giá trị phân số và cây tìm kiếm phân nhánh bị phình to hơn so với suy diễn logic trực tiếp của SAT. Đây là nguyên nhân chính khiến CBC (147.10 ms) chậm hơn hẳn một bậc độ lớn.



Hình 1: So sánh thời gian Encode vs Solve và quy mô số biến phụ/mệnh đề giữa 5 kỹ thuật SAT Encodings trên tập câu đố chuẩn hóa.



Hình 2: Xu hướng biến thiên tổng thời gian (mã hóa + giải) của 5 kỹ thuật SAT Encoding theo các cấp độ khó (Sudoku 9x9).



Hình 3: Biểu đồ đối chuẩn thời gian giải giữa SAT Solver (Glucose3) và các bộ giải CP, MIP hàng đầu.

6. Nghiên cứu trường hợp thực nghiệm (Experimental Case Studies)

Các kết quả ở Bảng 2–3 và Hình 1–3 là số liệu trung bình trên toàn tập 299 câu đố. Để làm rõ cơ chế hoạt động ở mức vi mô và kiểm chứng chéo các kết luận nêu trên, mục này phân tích chi tiết ba trường hợp thực nghiệm: (i) theo kích thước bảng, (ii) theo cấp độ khó và (iii) theo kỹ thuật mã hóa.

Giao thức đo lường. Câu đố đại diện cho mỗi nhóm được chọn là câu có số gợi ý bằng *trung vị* của nhóm (nhằm tránh các trường hợp ngoại lệ). Mỗi cấu hình (câu đố × phương pháp mã hóa) được lặp lại 60 lần; thời gian mã hóa và thời gian giải lấy theo **giá trị nhỏ nhất** (min-of-60) để loại bỏ nhiễu do bộ thu gom rác (Garbage Collector) của Python; sau đó lặp lại toàn bộ thí nghiệm trong 3 phiên độc lập và lấy **trung vị** giữa các phiên. Tất cả các lần chạy đều cho nghiệm hợp lệ (đã kiểm tra lại toàn bộ hàng, cột, khối và mọi gợi ý ban đầu). Do CPU thử nghiệm (Intel Core i5-1235U) liên tục thay đổi tần số (turbo boost), thời gian mã hóa dao động khoảng $\pm 30\%$ giữa các phiên đo, vì vậy các giá trị thời gian chỉ mang tính đại diện định tính; ngược lại, **số biến và số mệnh đề là các hằng số tuyệt đối**, lặp lại chính xác qua mọi lần chạy.

6.1. Trường hợp theo kích thước bảng (4×4, 9×9, 16×16)

Ba câu đố đại diện lần lượt là `matrix4x4_0002` (4×4, 4 gợi ý), `norvig_top95_0053` (9×9, 26 gợi ý) và `hexadoku16x16_0005` (16×16, 77 gợi ý). Bảng 4 tổng hợp quy mô CNF và thời gian tương ứng.

Nhận xét:

- **Quy luật tăng trưởng**: số mệnh đề của Pairwise bám sát hàm bậc hai và vượt mốc 120.000 mệnh đề ở 16×16 ($123,981$), gấp 2.69 lần 2-Product ($46,157$) — tương ứng mức giảm 63%; so với Sequential Counter ($46,157$ mệnh đề) mức tăng cũng là 2.69 lần. Ba kỹ thuật tuyến tính (Commander, 2-Product, Sequential Counter) chỉ sinh 46–52 nghìn mệnh đề nên kiểm soát tốt hiện tượng bùng nổ.

- **Khoảng cách thời gian nới rộng theo kích thước**: ở 4×4 cả 5 phương pháp đều dưới 0.8 ms; ở 9×9 khoảng cách còn nhỏ (2.93–7.34 ms); đến 16×16 khác biệt trở nên rõ rệt khi 2-Product (17.87 ms) nhanh hơn Pairwise (34.22 ms) và Sequential Counter (34.26 ms) khoảng **1.9 lần**, nhanh hơn Binary (32.92 ms) khoảng 1.8 lần.

- **Trường hợp đặc biệt** 4×4: với $n = 4$ quá nhỏ, 2-Product lại sinh *nhiều mệnh đề hơn* Pairwise (708 so với 452) vì chi phí sinh lưới chi phối: $2n + \binom{p}{2} + \binom{q}{2} = 2 \cdot 4 + 1 + 1 = 10$ mệnh đề cho mỗi ràng buộc Exactly-One, lớn hơn $\binom{4}{2} = 6$ của Pairwise. Kết quả này cho thấy các encoding tuyến tính chỉ phát huy ưu thế từ khoảng $n \geq 9$ trở lên.

- **Chi phí mã hóa luôn chi phối**: ở cả ba kích thước, thời gian giải thuần túy đều nhỏ hơn thời gian mã hóa từ khoảng 3 đến 6 lần, khẳng định lại nhận định ở Mục 5.2.

6.2. Trường hợp theo cấp độ khó

Hai bảng 5 và 6 trình bày kết quả trên bốn câu đố 9×9 đại diện cho bốn cấp độ khó, với số gợi ý giảm dần từ 36 (Easy) xuống 23 (Expert).

Nhận xét:

- **Tổng thời gian tăng nhẹ và gần như đơn điệu** khi độ khó tăng: 2-Product từ 3.27 lên 4.90 ms (+50%), Sequential Counter từ 5.00 lên 7.37 ms (+47%) và Pairwise từ 3.23 lên 4.13 ms (+28%). Biên độ tuyệt đối giữa Easy và Expert chỉ dưới 2.5 ms nên vẫn khẳng định số gợi ý ảnh hưởng không lớn tới tổng thời gian.

- **Nguyên nhân nằm ở pha giải**: thời gian giải thuần túy tăng từ 2.1 đến 5.0 lần khi đi từ Easy (36 gợi ý) đến Expert (23 gợi ý): Pairwise 0.32 → 0.67 ms; Binary 0.48 → 1.14 ms; Commander 0.32 → 1.52 ms; 2-Product 0.33 → 1.60 ms; Sequential Counter 0.21 → 1.36 ms. Khi số gợi ý giảm, ít mệnh đề đơn nguyên được nạp trước nên khả năng lan truyền đơn vị giảm và solver phải thực hiện nhiều bước phân nhánh hơn. Trong khi đó, pha mã hóa gần như độc lập với độ khó vì chỉ phụ thuộc vào N và số gợi ý.

- **Liên hệ với Hình 2**: hình đó biểu diễn *trung bình cộng* trên toàn bộ câu đố mỗi cấp (dễ bị chi phối bởi các ngoại lệ thời gian mã hóa do GC, có lần lên tới ~ 35 ms) nên đường cong có đỉnh tại cấp *Hard*; hai bảng trên dùng *trung vị* min-of-60 trên một câu đại diện nên cho xu hướng tăng đơn điệu. Cả hai quan sát hội tụ về cùng một kết luận: chênh lệch tổng thời gian giữa 4 cấp độ chỉ vài mili-giây, và phần chênh lệch chủ yếu nằm ở pha giải chứ không phải pha mã hóa.

Bảng 4
Quy mô CNF và thời gian trên ba câu đố đại diện theo kích thước bảng (min-of-60, trung vị của 3 phiên đo). Ký hiệu: Seq. Counter = Sequential Counter.

Bảng	Puzzle	Gợi ý	Mã hóa AMO	Biến	Mệnh đề	Enc. (ms)	Solve (ms)	Tổng (ms)
4×4	`matrix4x4_0002`	4	Pairwise	64	452	0.17	0.05	0.22
			Binary	192	580	0.58	0.07	0.65
			Commander	128	580	0.34	0.07	0.41
			2-Product	320	708	0.33	0.08	0.41
			Seq. Counter	256	580	0.73	0.06	0.79
9×9	`norvig_top95_0053`	26	Pairwise	729	12,014	2.80	0.45	3.25
			Binary	2,025	12,014	6.81	0.53	7.34
			Commander	1,701	8,126	3.05	0.43	3.48
			2-Product	2,673	8,126	2.50	0.43	**2.93**
			Seq. Counter	3,321	7,802	5.50	0.38	5.88
16×16	`hexadoku16x16_0005`	77	Pairwise	4,096	123,981	30.12	4.10	34.22
			Binary	8,192	66,637	30.14	2.78	32.92
			Commander	11,264	52,301	20.51	3.26	23.77
			2-Product	12,288	46,157	15.63	2.24	**17.87**
			Seq. Counter	19,456	46,157	30.73	3.53	34.26

Bảng 5
Tổng thời gian (mã hóa + giải, ms) trên bốn câu đố 9×9 đại diện: `norvig_easy50_0005` (Easy), `norvig_easy50_0003` (Medium), `norvig_top95_0012` (Hard) và `norvig_hardest_0002` (Expert).

Độ khó	Gợi ý	Pairwise	Binary	Commander	2-Product	Seq. Counter
Easy	36	3.23	7.85	3.81	3.27	5.00
Medium	28	3.21	6.20	3.52	3.50	6.44
Hard	25	3.54	7.01	4.03	3.63	7.01
Expert	23	4.13	8.32	5.53	4.90	7.37

Bảng 6
Thời gian giải thuần túy (ms, chỉ tính pha `Glucose3.solve()`) trên cùng bốn câu đố 9×9 đại diện theo cấp độ khó.

Độ khó	Gợi ý	Pairwise	Binary	Commander	2-Product	Seq. Counter
Easy	36	0.32	0.48	0.32	0.33	0.21
Medium	28	0.34	0.40	0.27	0.33	0.26
Hard	25	0.47	0.63	0.66	0.42	0.78
Expert	23	0.67	1.14	1.52	1.60	1.36

6.3. Trường hợp theo kỹ thuật mã hóa

Bảng 7 phân rã cấu trúc file CNF của câu đố 9×9 đại diện (26 gợi ý) theo từng kỹ thuật mã hóa.
Nhận xét:

- **Kiểm chứng công thức lý thuyết**: mỗi bảng 9×9 sinh chính xác $4N^2 = 324$ ràng buộc Exactly-One (hàng, cột, khối, ô), tương ứng 324 mệnh đề ALO và 26 mệnh đề đơn nguyên từ gợi ý. Số mệnh đề AMO cho *mỗi* ràng buộc Exactly-One khớp chính xác với Mục 3: Pairwise $36 = \binom{9}{2}$; Binary $36 = 9\lceil \log_2 9 \rceil = 9 \times 4$; Commander 24; 2-Product $24 = 2 \times 9 + \binom{3}{2} + \binom{3}{2}$; Sequential Counter $23 = 3 \times 9 - 4$. Vì vậy Commander và 2-Product cắt 33% số mệnh đề AMO, Sequential Counter cắt 36%; tính trên tổng số mệnh đề, mức giảm lần lượt là 32% và 35% so với Pairwise.
- **Kiểm chứng số biến phụ**: các giá trị đo được cũng đúng bằng công thức: Binary $1,296 = 324 \times \lceil \log_2 9 \rceil$; Commander $972 = 324 \times \lceil 9/3 \rceil$; 2-Product $1,944 = 324 \times (p + q) = 324 \times 6$; Sequential Counter $2,592 = 324 \times (n - 1)$. Điều này xác nhận các cài đặt trong mã nguồn triển khai đúng đặc tả lý thuyết.

Bảng 7
Phân rã cấu trúc CNF của câu đố `norvig_top95_0053` (9×9, 26 gợi ý): 324 ràng buộc Exactly-One ($4N^2$) tương ứng 324 mệnh đề ALO, 26 mệnh đề đơn nguyên từ gợi ý, phần còn lại là AMO.

Phương pháp	Biến	Biến phụ	Mệnh đề	ALO	AMO	Gợi ý	Encode (ms)	Solve (ms)
Pairwise (Naive)	729	0	12,014	324	11,664	26	2.80	0.45
Binary (Bitwise)	2,025	1,296	12,014	324	11,664	26	6.81	0.53
Commander	1,701	972	8,126	324	7,776	26	3.05	0.43
2-Product	2,673	1,944	8,126	324	7,776	26	2.50	0.43
Sequential Counter	3,321	2,592	7,802	324	7,452	26	5.50	0.38

Bảng 8
Câu đố 9×9 đại diện `norvig_top95_0053` (26 gợi ý) và nghiệm do Glucose3 với mã hóa 2-Product tìm được.

Đề bài

	2			5	9	3		
8			5	4	6			
9	4					8		
		2		3				
	6			8		7	3	
7			2					
				4		3	8	
	7					6		
								5

Nghiệm

1	2	6	4	7	8	5	9	3
8	3	7	5	9	2	4	6	1
9	4	5	3	6	1	2	7	8
4	1	2	9	3	7	8	5	6
5	6	9	1	8	4	7	3	2
7	8	3	2	5	6	9	1	4
2	5	1	6	4	9	3	8	7
3	7	4	8	1	5	6	2	9
6	9	8	7	2	3	1	4	5

- **Thời gian**: 2-Product cho thời gian mã hóa nhỏ nhất (2.50 ms) và cũng nhỏ nhất ở pha giải (0.43 ms). Binary (6.81 ms) và Sequential Counter (5.50 ms) chậm nhất ở pha mã hóa do cả hai đều gọi thư viện PySAT để sinh mệnh đề, trong khi Pairwise, Commander và 2-Product được cài đặt trực tiếp bằng Python nên ít chi phí gọi hàm hơn.

Bảng 8 minh họa một trường hợp cụ thể: đề bài 9×9 có 26 gợi ý và nghiệm duy nhất do Glucose3 (mã hóa 2-Product) tìm được, đã được kiểm tra thỏa mãn toàn bộ 5 nhóm ràng buộc của bài toán.

7. Kết luận và Hướng phát triển (Conclusion)

Bài báo đã hoàn thành nghiên cứu toàn diện về việc mô hình hóa logic và giải bài toán Sudoku tổng quát $N \times N$ bằng SAT Encoding và đối chuẩn với các công cụ CP/MIP:

- Triển khai hoàn chỉnh 5 kỹ thuật mã hóa ràng buộc số lượng At-Most-One: Pairwise, Binary, Commander, 2-Product, và Sequential Counter.
- Chứng minh thực nghiệm rằng **2-Product Encoding** đạt sự cân bằng tốt nhất giữa số biến phụ ($O(\sqrt{n})$), số mệnh đề sinh ra và tổng thời gian xử lý (2.8–4.1 ms trên mọi cấp độ khó); trong khi **Sequential Counter** sinh ít mệnh đề nhất nhưng tốn nhiều biến phụ và thời gian mã hóa nhất.
- Khẳng định lợi thế về tốc độ của tiếp cận **SAT Solving** đối với lớp bài toán logic tổ hợp hữu hạn so với các bộ giải tối ưu hóa toán học truyền thống (ILP/MIP), dù các bộ giải MIP thương mại (Gurobi) đã bám sát khoảng cách rất nhỏ.

Hướng phát triển trong tương lai: Nghiên cứu mở rộng sang các biến thể Sudoku phức tạp hơn (như Killer Sudoku, Diagonal Sudoku, Greater-Than Sudoku) đòi hỏi kết hợp kỹ thuật thỏa mãn modulo lý thuyết (SMT - Satisfiability Modulo Theories) và tối ưu hóa thỏa mãn lớn nhất (MaxSAT).

Data Availability (Tính sẵn sàng của dữ liệu)

Toàn bộ mã nguồn chương trình, bộ dữ liệu 299 câu đố benchmark chuẩn hóa, các script đo lường tự động và hình ảnh biểu đồ thực nghiệm chi tiết được công khai phục vụ cộng đồng học thuật tại GitHub repository:

https://github.com/nmson8206/Sudoku_prj

Tài liệu

Audemard, G., Simon, L., 2009. Predicting learnt clauses quality in modern sat solvers. IJCAI 9, 399–404.

Chen, J., 2010. A new sat encoding of the at-most-one constraint, in: Constraint Programming and Decision Making: Workshop at the European Conference on Artificial Intelligence (ECAI).

Frisch, A.M., Peugniez, N., Doggett, A.J., Nightingale, P., 2008. Effective propositional encodings of constraint satisfaction problems. Artificial Intelligence 172, 1961–1994.

Gurobi Optimization, LLC, 2024. Gurobi optimizer reference manual. Online: https://www.gurobi.com .

IBM Corporation, 2024. Ibm ilog cplex optimization studio cplex user's manual. Online: https://www.ibm.com/analytics/cplex-optimizer .

Klieber, W., Kwon, G., 2007. Efficient cnf encoding for selecting 1 from n objects, in: International Conference on Theory and Applications of Satisfiability Testing, Springer. pp. 208–214.

Norvig, P., 2006. Solving every sudoku puzzle. Online essay: http://norvig.com/sudoku.html .

Perron, L., Furnon, V., 2024. Google or-tools: Cp-sat solver reference. Google Developers .

Sinz, C., 2005. Towards an optimal cnf encoding of boolean cardinality constraints. Principles and Practice of Constraint Programming-CP 2005 , 827–831.